Stimulus and Ajax

Even though “the Hotwire way” is all about sending and receiving HTML between the client and the server, you may still need to receive plain data sometimes. For example, you may have a legacy API section of your code, or it may be useful to call a third-party API from your client. Let’s look at how we would make those API calls in our Stimulus code.

The simplest mechanism for making Ajax calls in modern JavaScript is with the fetch function and the async/await syntax for handling asynchronous behavior. The fetch method takes two arguments: the URL to contact, and an object containing optional arguments, like method for HTTP method, body, data, and so on.

The return value for fetch is a Response object wrapped in a JavaScript promise. A *promise* is a JavaScript object that represents an asynchronous process that will be executed eventually and might return data when it is done. Calling a method named then on the promise object allows a function to be invoked only when the process has ended successfully.

If you’ve written JavaScript code in the last few years, you might be familiar with this pattern, where multiple asynchronous actions result in promises that nest one after the other:

```
updateData() {
  fetch("/sold_out_concerts").then (response) => {
    response.text().then (text) => {
      // do something with text
    }
  }
}
```

In these cases where multiple asynchronous events need to happen, the sequence of then calls on the various promises can get complicated. The async and await keywords were introduced to simplify using asynchronous processes:

```
asnyc updateData() {
  const response = await fetch("/sold_out_concerts")
```